Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

`https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist`

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$\boxed{x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32\times28\times28\times1}.$$

Pixel values should be converted from

$$[0, 255] \to [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28\times28\times1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28\times28\times1}.$$

Students must display:

```
Original image
Reconstructed image
Reconstruction error
```

For the denoising model, display:

```
Original clean image
Noisy image
Denoised reconstruction
```

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.
The encoder maps the input to a lower-dimensional representation:

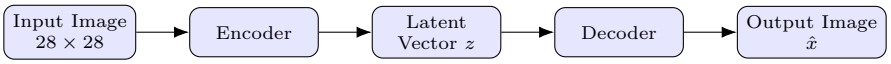
$$z = f_\theta(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_\phi(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2.$$



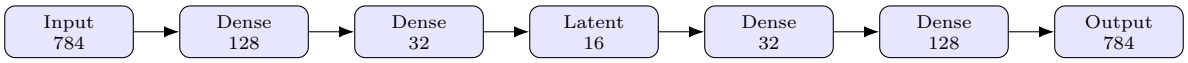
The latent vector is a compressed representation of the input.

7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.
Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.
Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

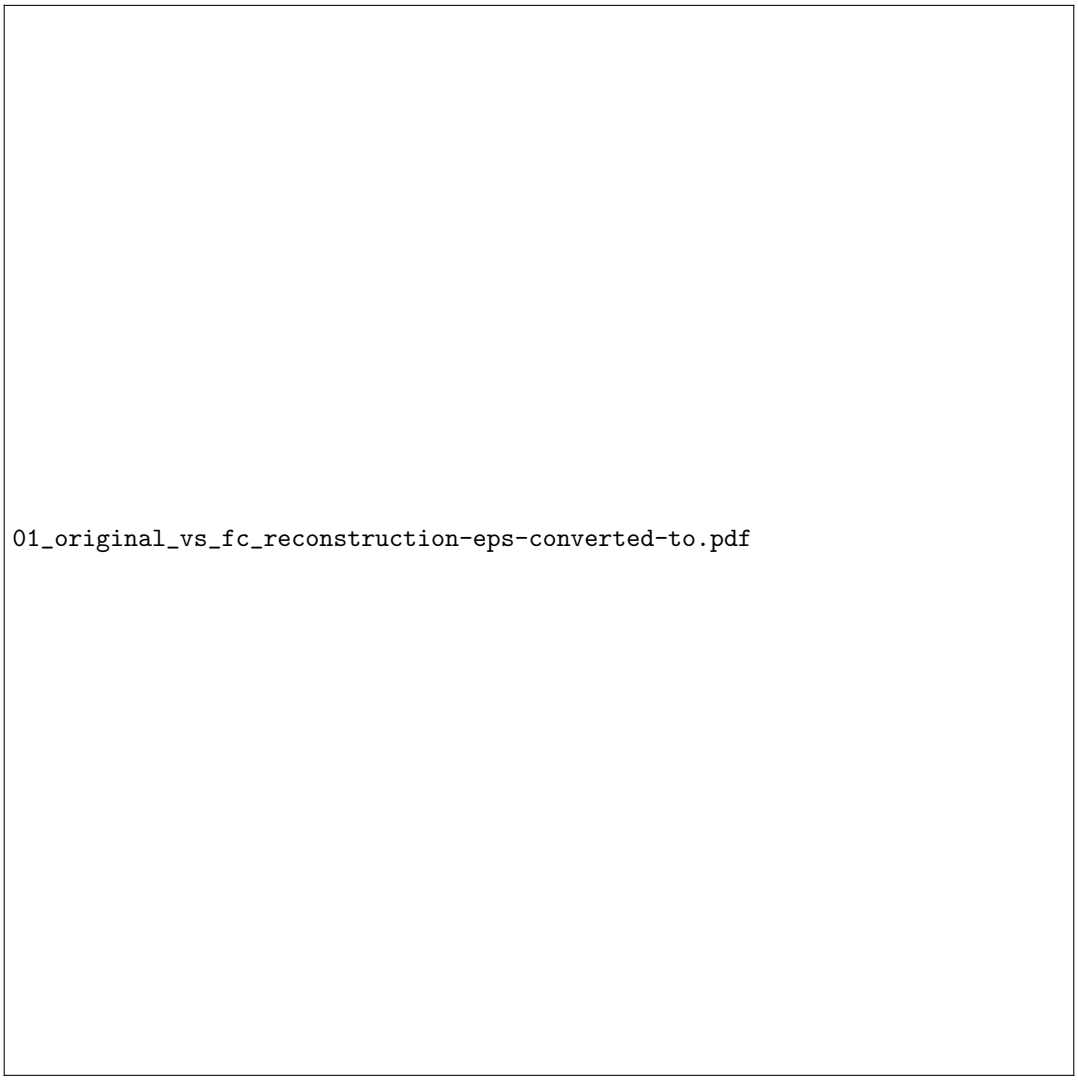


Figure 1: Original images versus Fully Connected Autoencoder reconstructions

Inference: The figure compares the original MNIST images with their reconstructed versions produced by the Fully Connected Autoencoder. The major digit shapes are preserved, although some fine details and sharper boundaries may be lost during reconstruction.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

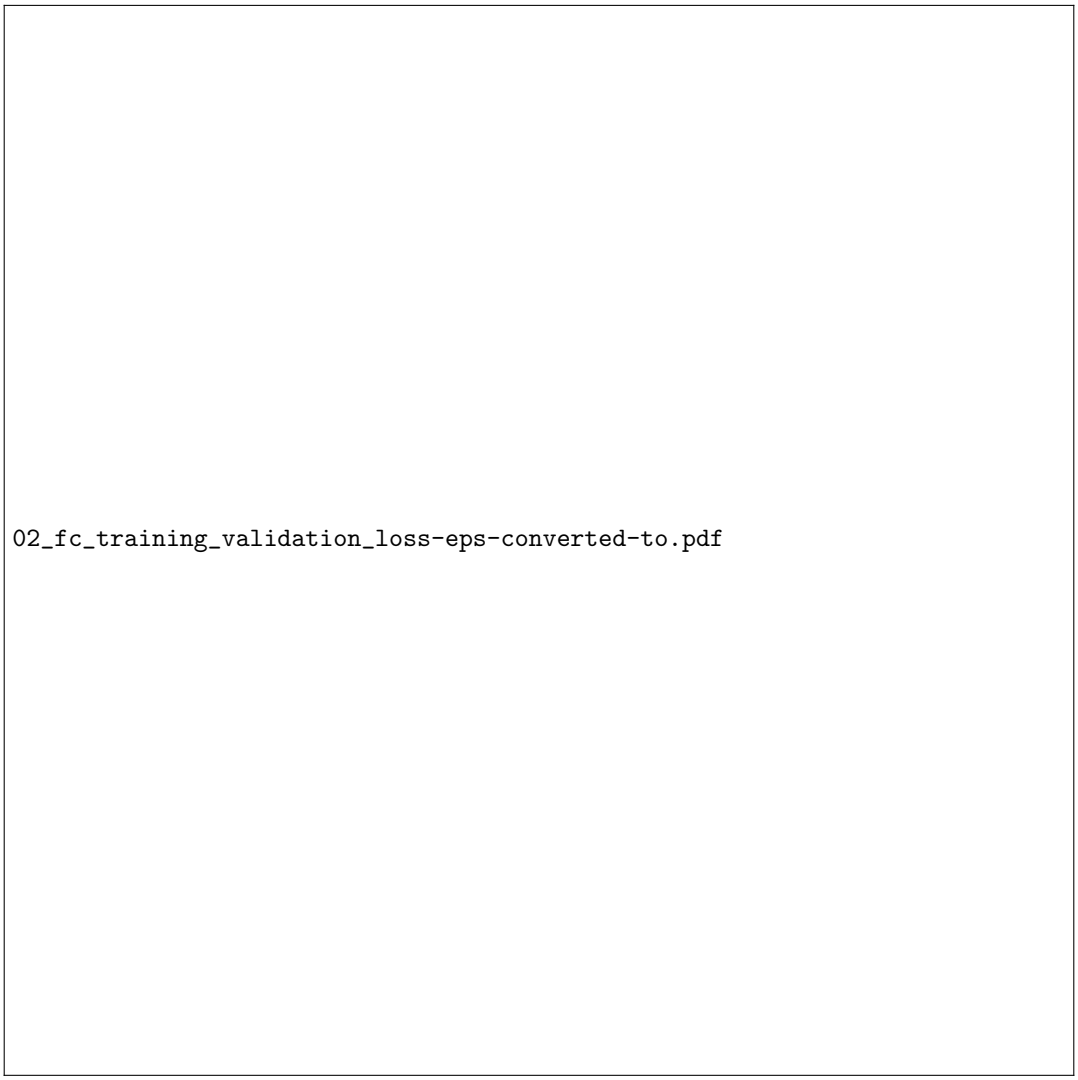


Figure 2: Training and validation reconstruction loss of the Fully Connected Autoencoder

Inference: The training and validation loss curves show how the Fully Connected Autoencoder learns over the epochs. Both losses decrease and gradually stabilize, indicating convergence, while a small gap between the curves indicates the extent of any overfitting.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.
The implementation may use:
`https://scikit-image.org/docs/stable/api/skimage.metrics.html`

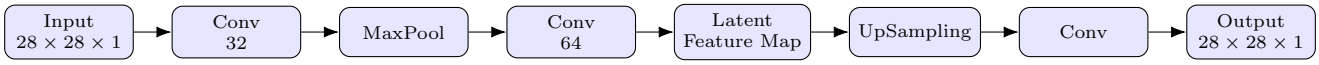
Report:

Table 1: Reconstruction metrics of the Fully Connected Autoencoder

Metric	Value
Test MSE	0.022454
Test MAE	0.058811
Mean SSIM	0.731017

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.
The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

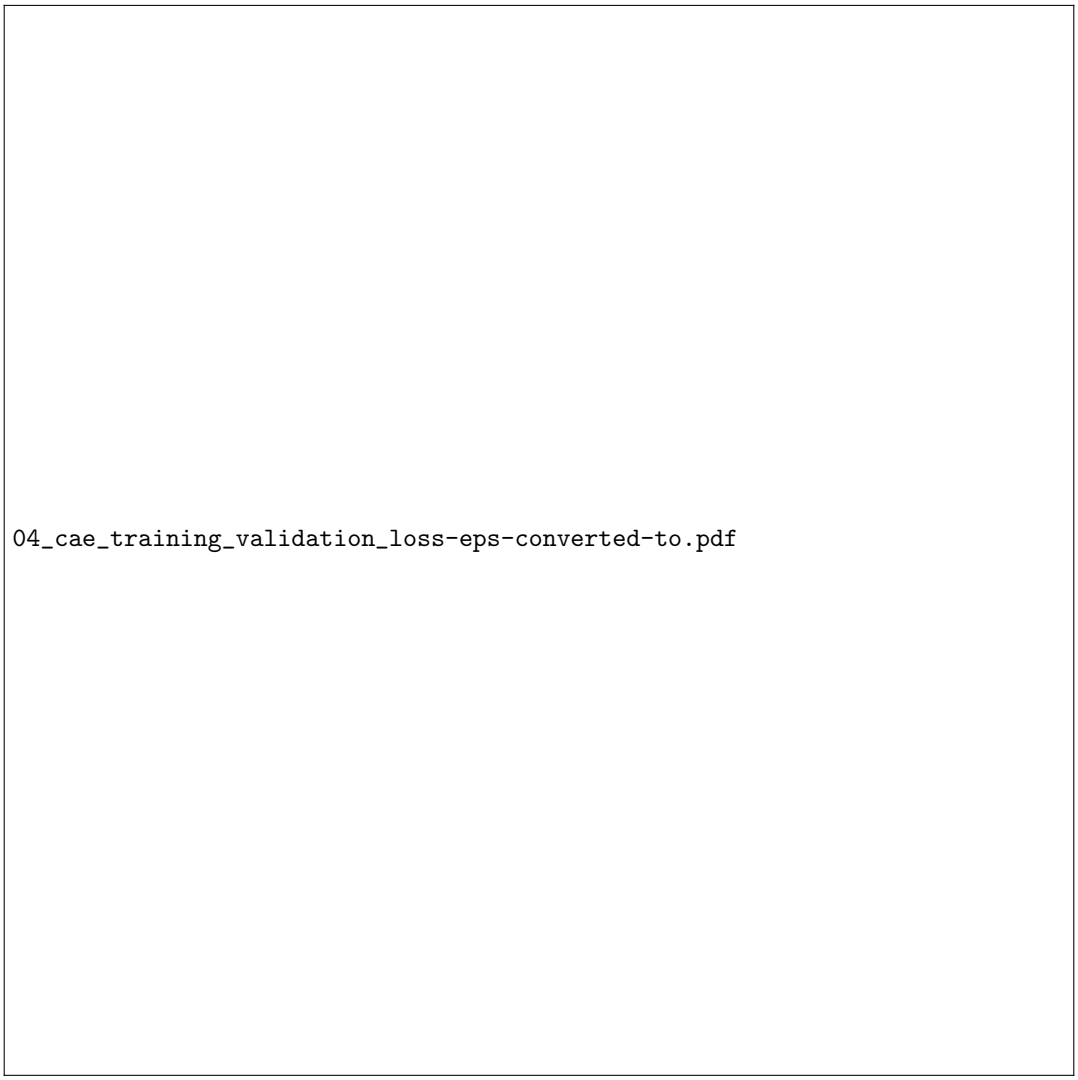


Figure 3: Training and validation reconstruction loss of the Convolutional Autoencoder

Inference: The training and validation loss curves demonstrate the learning behavior of the Convolutional Autoencoder. The decreasing losses indicate that the model progressively improves its reconstruction capability and approaches convergence during training.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Table 2: Comparison of Fully Connected and Convolutional Autoencoders

Model	**MSE**	**MAE**	**SSIM**	**Parameters**
Fully Connected AE	0.022454	0.058811	0.731017	211040
Convolutional AE	0.003355	0.017342	0.966246	74497

Plot 3: Reconstruction Comparison

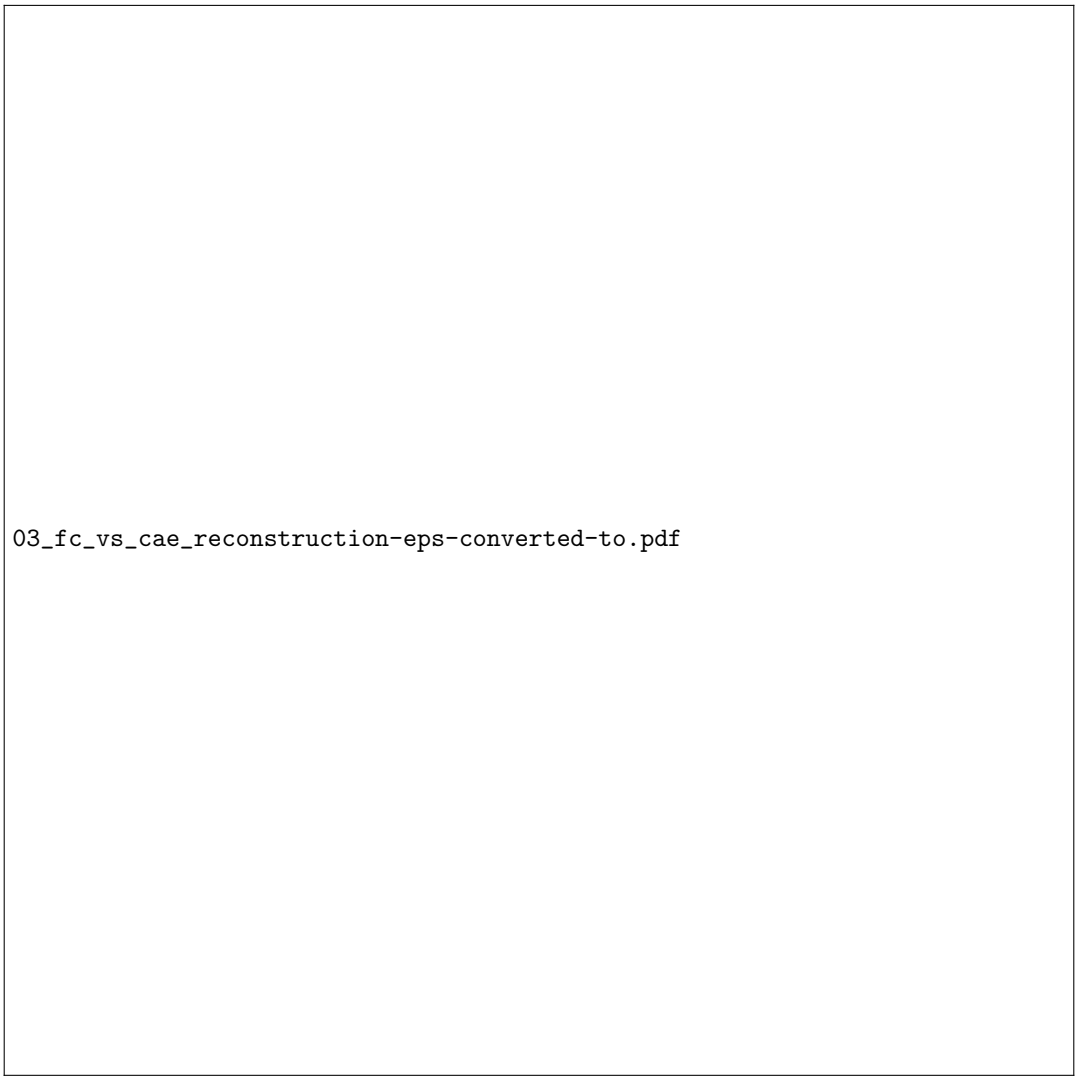


Figure 4: Reconstruction comparison between Fully Connected and Convolutional Autoencoders

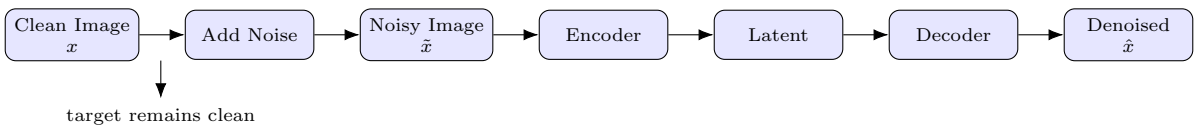
Inference: The figure compares the original image with reconstructions from the Fully Connected Autoencoder and Convolutional Autoencoder. The convolutional model preserves spatial and local image structure more effectively because convolutional layers retain the spatial relationships between neighboring pixels.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.
The training mapping is:

$$\boxed{\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}}$$

where $\tilde{x}$ is a noisy version of the clean image x.



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \qquad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.
Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised



Figure 5: Clean, noisy and denoised MNIST images

Inference: The figure shows the effect of noise corruption and subsequent denoising by the autoencoder. The denoised images recover the main structure and shape of the original digits, although some fine details may remain distorted because of the information lost during noise addition.

Plot 5: Noise Level vs. Reconstruction Quality

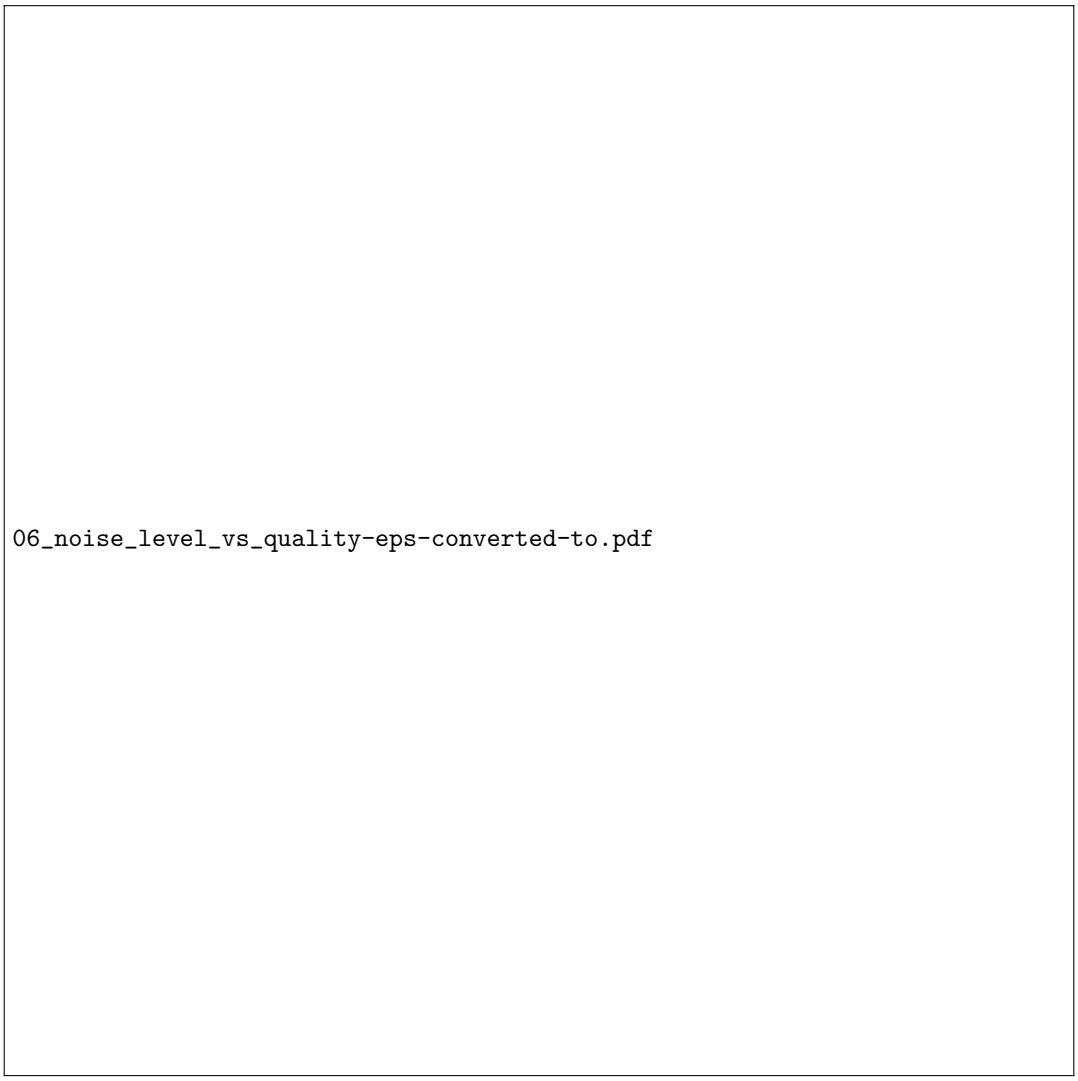


Figure 6: Effect of noise level on denoising reconstruction quality

Inference: The figure illustrates the relationship between Gaussian noise level and reconstruction quality using MSE, MAE and SSIM. Increasing the noise level makes reconstruction more challenging, generally increasing reconstruction error while reducing structural similarity between the original and denoised images.

Report:

Table 3: Effect of Gaussian noise level on denoising reconstruction quality

Noise Level	MSE	MAE	SSIM
0.1	0.007502	0.059067	0.664982
0.2	0.014070	0.094284	0.600739
0.3	0.024189	0.129131	0.544876

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

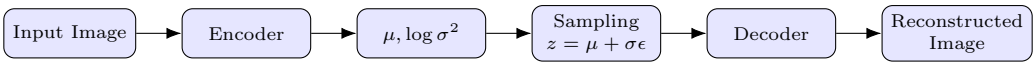
$$q_\phi(z|x) = \mathcal{N}\left(\mu(x), \mathrm{diag}(\sigma^2(x))\right).$$

The encoder produces:

$$\mu, \qquad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}\left(q_\phi(z|x) \parallel p(z)\right) = -\frac{1}{2}\sum_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.
Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

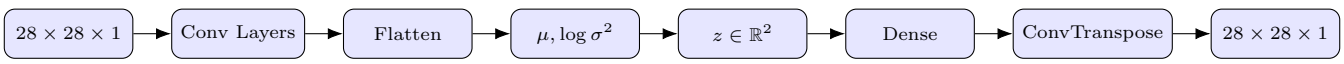
The encoder should produce:

$$\mu \in \mathbb{R}^2, \qquad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \to 28 \times 28 \times 1.$$

A compact architecture is:



18. Exploring the VAE Latent Space

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

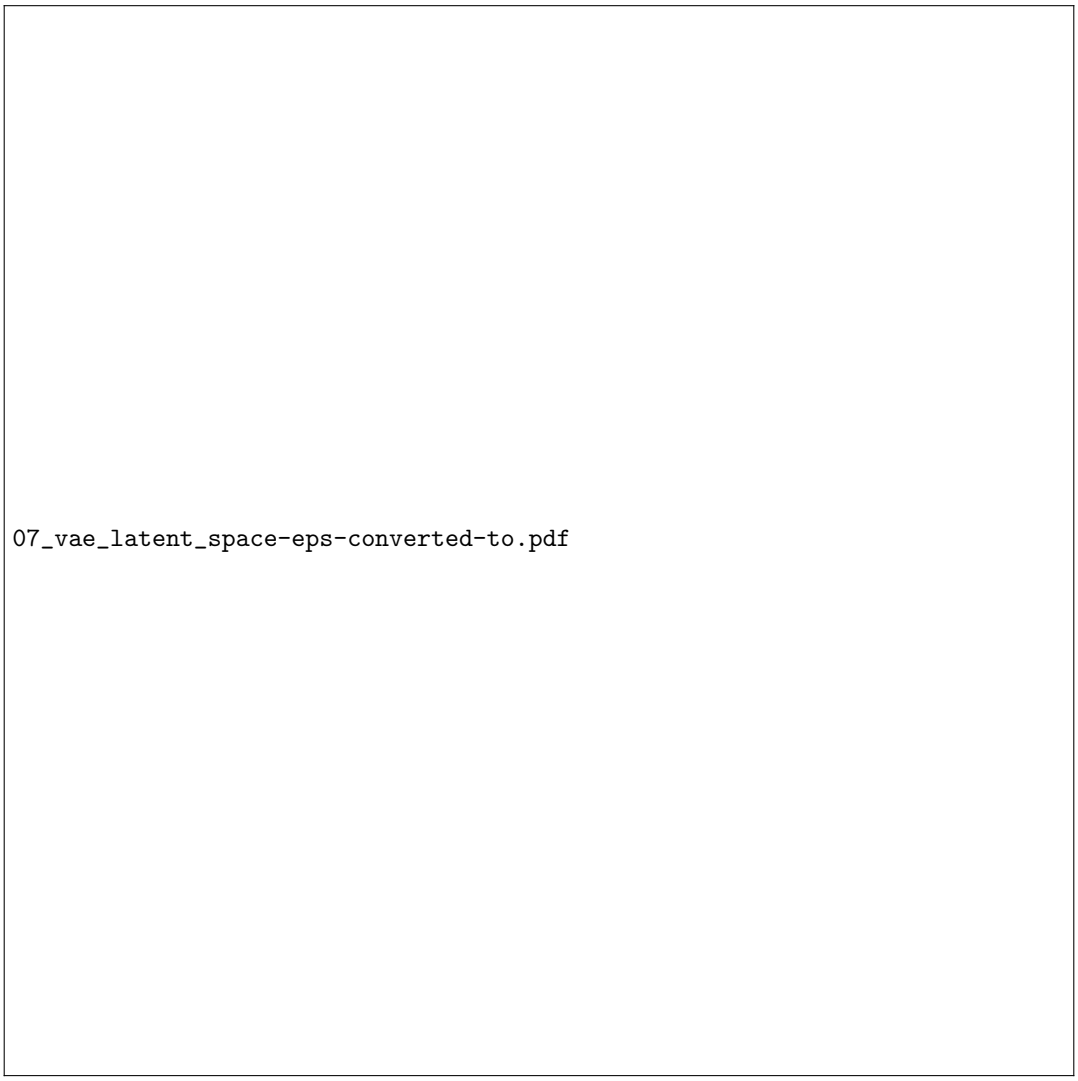


Figure 7: Two-dimensional VAE latent space visualization of MNIST digits

Inference: The latent-space visualization shows how MNIST images are represented in the two-dimensional VAE latent space. Images belonging to similar digit classes tend to form groups, indicating that the VAE has learned meaningful representations of the input data.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

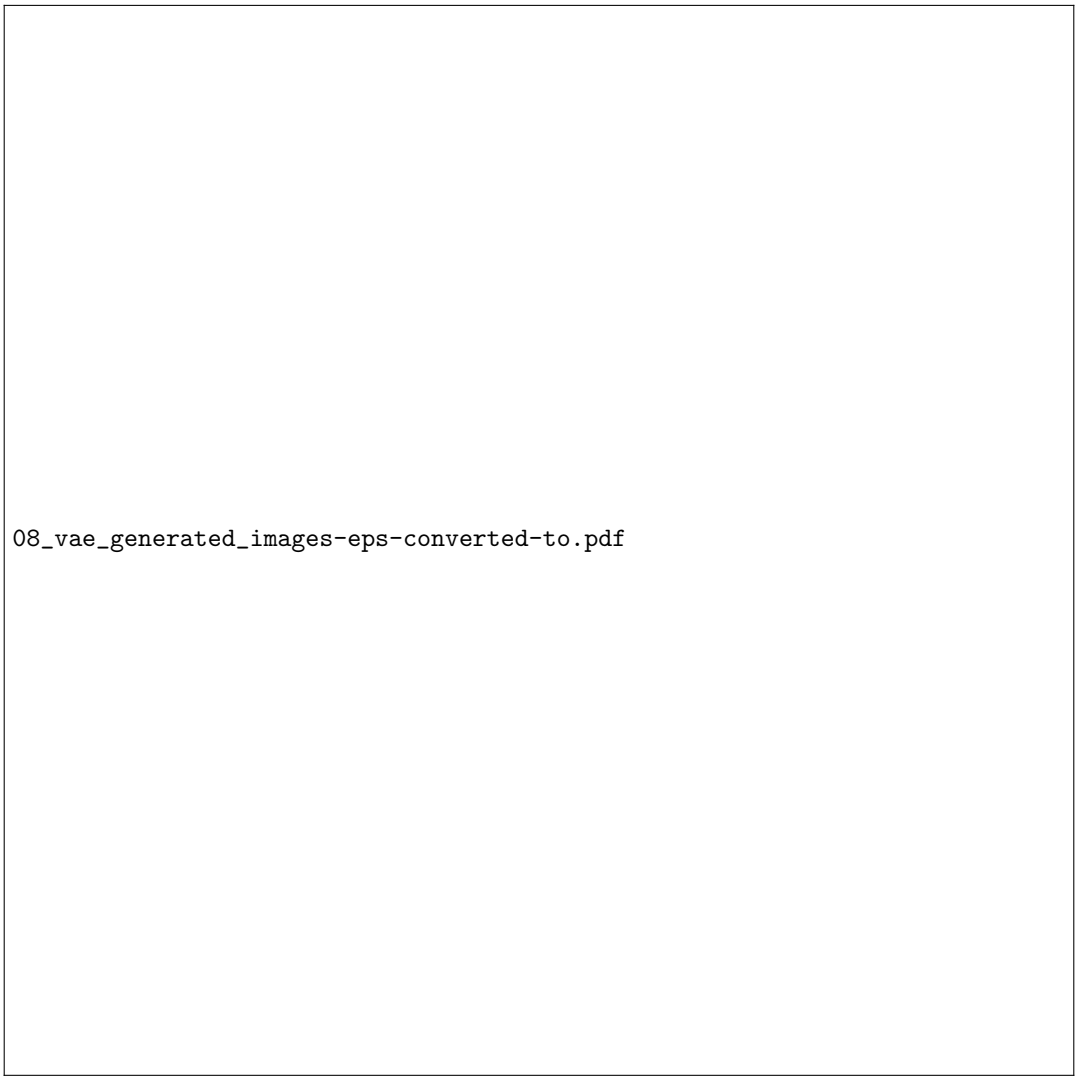


Figure 8: Images generated by sampling from the VAE latent space

Inference: The figure shows new MNIST-like images generated by sampling points from the VAE latent space. The generated samples demonstrate that the VAE can learn the underlying distribution of handwritten digits and produce new samples from the learned representation.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \qquad z_B.$$

Construct

$$z(\alpha) = (1-\alpha)z_A + \alpha z_B, \qquad \alpha \in [0,1].$$

Use values such as:

$$\alpha = 0,\ 0.1,\ 0.2, \ldots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

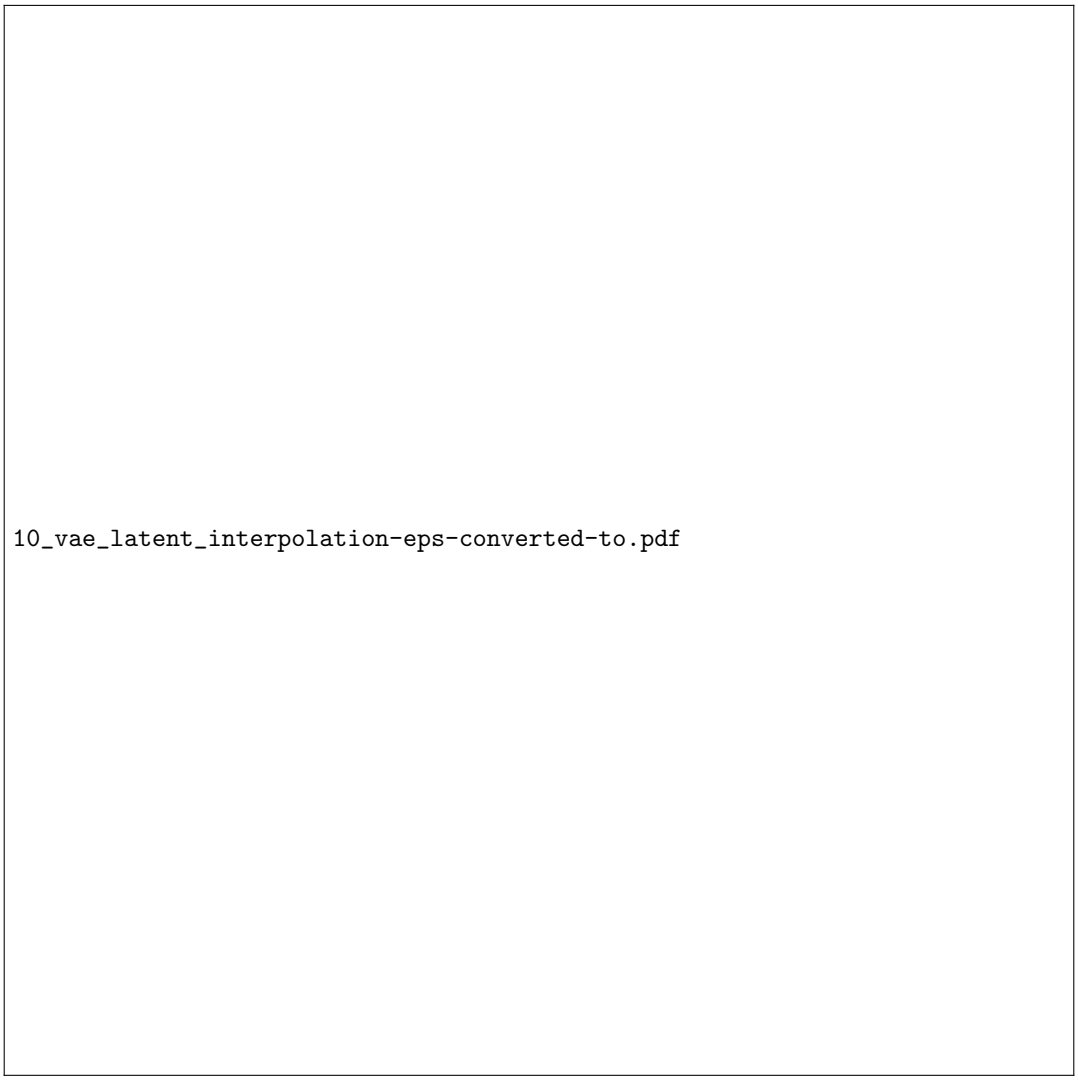


Figure 9: Latent space interpolation between two VAE latent representations

Inference: The interpolation demonstrates that the VAE latent space provides a smooth transition between two encoded samples. The generated images gradually change from the characteristics of the first sample toward those of the second sample, showing the continuity of the learned latent representation.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Table 4: Reconstruction comparison across autoencoder models

Model	**MSE**	**MAE**	**SSIM**	**Parameters**
Fully Connected AE	0.022454	0.058811	0.731017	211040
Convolutional AE	0.003355	0.017342	0.966246	74497
Denoising CAE	0.014070	0.094284	0.600739	74497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

Table 5: Variational Autoencoder evaluation metrics

VAE Metric	Value
Reconstruction Loss	154.573807
KL Loss	5.339083
Total Loss	159.912872
Test MSE	0.046313
Test MAE	0.107126
Mean SSIM	0.462673

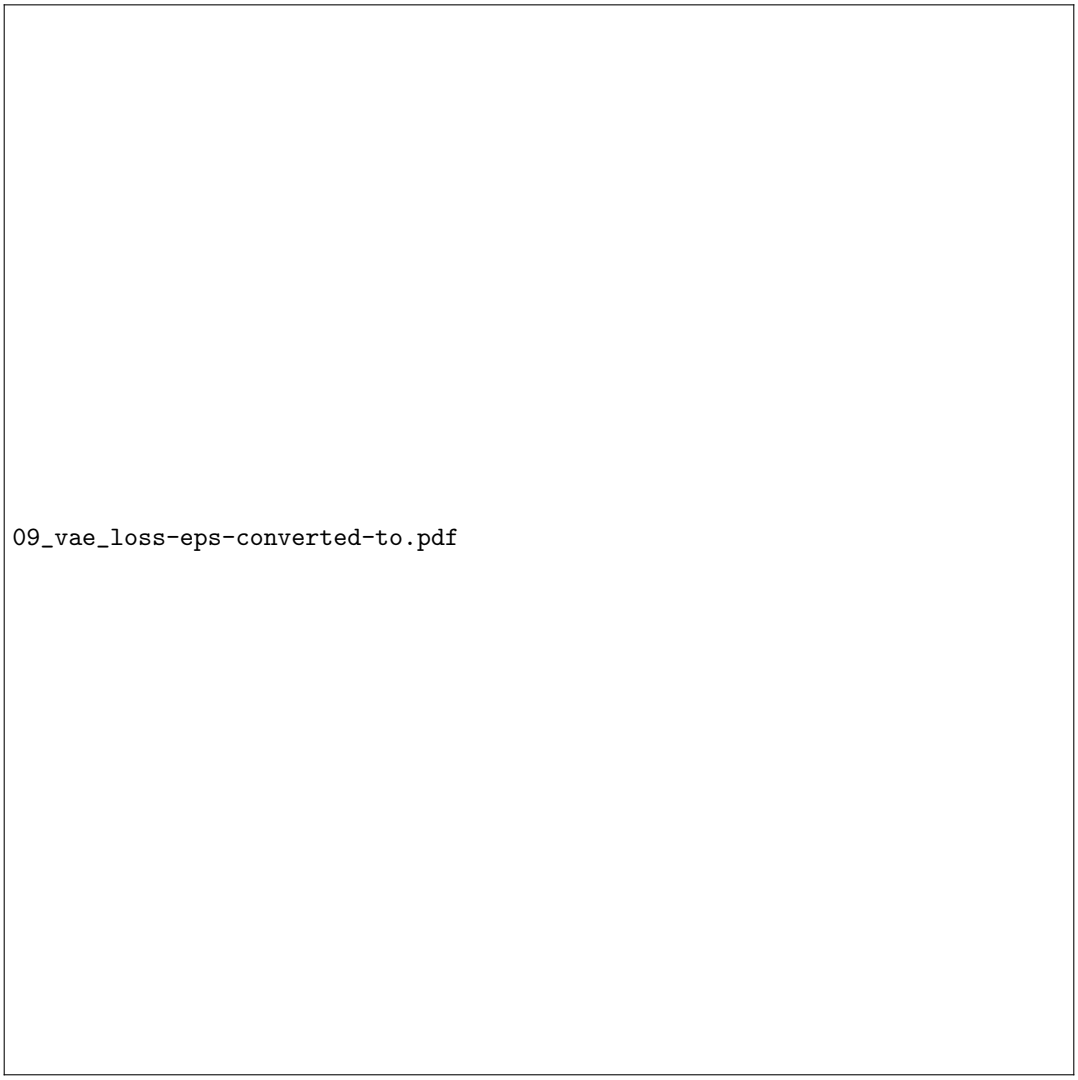


Figure 10: VAE training and validation loss curves

Inference: The VAE loss curves show the change in total loss during training. The loss generally decreases as training progresses, indicating that the model learns to reconstruct the input images while simultaneously regularizing the latent space through the KL-divergence term.

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.

7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D}\sum_{j=1}^{D}(x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

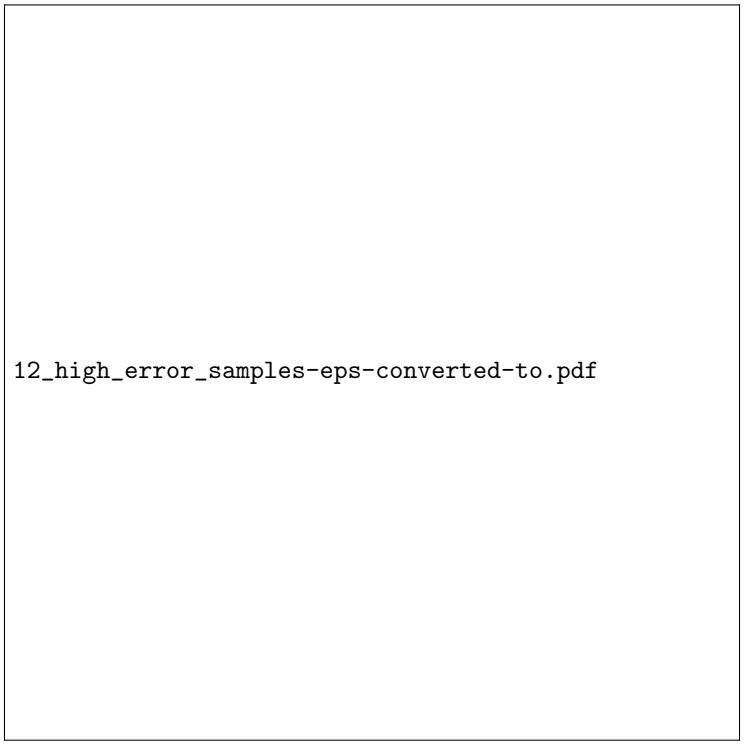


Figure 11: Five test images with the highest reconstruction errors

Inference: The figure shows the five test images with the highest reconstruction errors and their corresponding VAE reconstructions. These samples are more difficult to reconstruct because their shapes or structures are less accurately represented in the learned latent space.

25. Consolidated Results

Table 6: Consolidated quantitative results of the autoencoder models

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC Autoencoder	0.022454	0.058811	0.731017	211040	23.80415
Conv. Autoencoder	0.003355	0.017342	0.966246	74497	346.9148
Denoising CAE	0.014070	0.094284	0.600739	74497	278.0276

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Table 7: Effect of latent dimension on reconstruction quality

Latent Dimension	MSE	SSIM	Parameters
4	0.024128	0.746240	22597
8	0.014255	0.852235	26057
16	0.012338	0.871739	32977
32	0.009835	0.899599	46817

28. Discussion Questions

1. What is an autoencoder?
   Ans: An autoencoder is a neural network that learns to reconstruct its input. It compresses the input into a smaller latent representation and then reconstructs the original data from it.
2. What is the purpose of the encoder?
   Ans: The encoder converts the input into a compact latent representation. It learns the most important features of the input while reducing its dimensionality.
3. What is the purpose of the decoder?
   Ans: The decoder takes the latent representation and tries to reconstruct the original input. Its goal is to produce an output as close as possible to the original data.
4. What is a latent representation?
   Ans: A latent representation is the compact form of the input produced by the encoder. It contains the important features needed to reconstruct the original data.

5. Why is the latent representation usually smaller than the input?
   Ans: A smaller latent representation forces the autoencoder to learn the most important information instead of simply copying the input. This also helps in dimensionality reduction and feature learning.
6. What happens when the latent dimension is too small?
   Ans: When the latent dimension is too small, the model cannot store enough information about the input. As a result, important details are lost and the reconstruction quality decreases.
7. What happens when the latent dimension is increased?
   Ans: Increasing the latent dimension gives the model more capacity to store information about the input. This can improve reconstruction quality, although too much capacity may reduce the compression benefit.
8. Why can a convolutional autoencoder preserve spatial information more effectively?
   Ans: A convolutional autoencoder processes nearby pixels together and preserves their spatial relationships. This makes it suitable for images, where edges, shapes and local patterns are important.
9. Differentiate an autoencoder and a denoising autoencoder.
   Ans: A normal autoencoder learns to reconstruct the original input from the original input. A denoising autoencoder receives a noisy image but is trained to reconstruct the corresponding clean image.
10. Why is the clean image used as the target in denoising?
    Ans: The clean image provides the correct reconstruction that the model should learn to produce. This teaches the autoencoder to remove noise while retaining the important features of the original image.
11. What happens when the noise level is increased?
    Ans: Increasing the noise level makes the original image harder to recover. Usually, reconstruction error increases and image similarity decreases as more noise is introduced.
12. Why are MSE, MAE and SSIM useful for image reconstruction?
    Ans: MSE and MAE measure the numerical difference between the original and reconstructed images. SSIM focuses more on structural similarity, so together these metrics provide a broader view of reconstruction quality.
13. What is a Variational Autoencoder?
    Ans: A Variational Autoencoder (VAE) is a generative model that learns a probability distribution in the latent space. It can use this learned distribution to reconstruct existing images and generate new ones.
14. How does a VAE differ from a conventional autoencoder?
    Ans: A conventional autoencoder usually maps an input to a single latent vector. A VAE learns the parameters of a latent probability distribution and samples from that distribution during reconstruction and generation.
15. Why does a VAE learn a distribution instead of a single latent vector?
    Ans: Learning a distribution makes the latent space smoother and more continuous. This allows the model to sample new latent points and generate new images that are similar to the training data.
16. What is the reparameterization trick?
    Ans: The reparameterization trick allows the VAE to sample a latent vector while still allowing gradients to flow through the network. It represents the sample using the learned mean, variance and a random value.
17. Why is ϵ sampled from a standard normal distribution?
    Ans: Sampling ϵ from a standard normal distribution provides a consistent source of random variation. The encoder can then control the final latent distribution using its learned mean and variance.
18. What is the role of KL divergence in the VAE loss?
    Ans: KL divergence encourages the learned latent distribution to remain close to the chosen prior distribution. This regularizes the latent space and makes it easier to sample meaningful new points.
19. What is the prior distribution used in the VAE?
    Ans: The VAE commonly uses a standard normal distribution, $\mathcal{N}(0, I)$, as its prior. The KL-divergence term encourages the learned latent distribution to stay close to this prior.
20. Why is a two-dimensional latent space useful for visualization?
    Ans: A two-dimensional latent space can be directly plotted on a graph. This makes it easier to observe how different digit classes are distributed and whether similar images form meaningful groups.
21. What does latent-space interpolation demonstrate?
    Ans: Latent-space interpolation shows whether the learned representation changes smoothly between two points. The generated images should gradually transition from the characteristics of one sample to another.
22. Why can a VAE generate new images?
    Ans: A VAE learns a structured probability distribution in its latent space. By sampling a new point from this distribution and passing it through the decoder, the model can generate a new image.
23. Why might some VAE-generated images be unclear?
    Ans: The sampled latent point may not correspond to a region that represents a clear training example. Also, limited training, a small latent dimension or strong KL regularization can lead to less detailed reconstructions.
24. What is the difference between reconstruction and generation?
    Ans: Reconstruction means recovering an image from its encoded representation. Generation means creating a new image by sampling a latent representation and passing it through the decoder.
25. Why should test images remain unseen during training?
    Ans: Keeping test images unseen provides a fair way to measure how well the model generalizes to new data. If

the model trains on the test images, the evaluation would no longer represent its performance on unseen inputs.

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

Table 8: Gaussian vs Salt-and-Pepper Noise

Noise Type	**MSE**	**MAE**	**SSIM**
Gaussian	0.005634	0.024374	0.929799
Salt-and-Pepper	0.006903	0.026809	0.915682

Table 9: Effect of Two Gaussian Noise Levels

Noise Sigma	**MSE**	**MAE**	**SSIM**
0.1	0.003348	0.018128	0.959326
0.3	0.007981	0.029981	0.897907

Table 10: Upsampling vs Conv2DTranspose Decoder

Decoder	**MSE**	**MAE**	**SSIM**	**Training Time (s)**
UpSampling2D + Conv2D	0.006368	0.025721	0.934999	106.900566
Conv2DTranspose	0.008835	0.031435	0.909001	63.467386

Plot 10: Latent Dimension vs. Reconstruction MSE

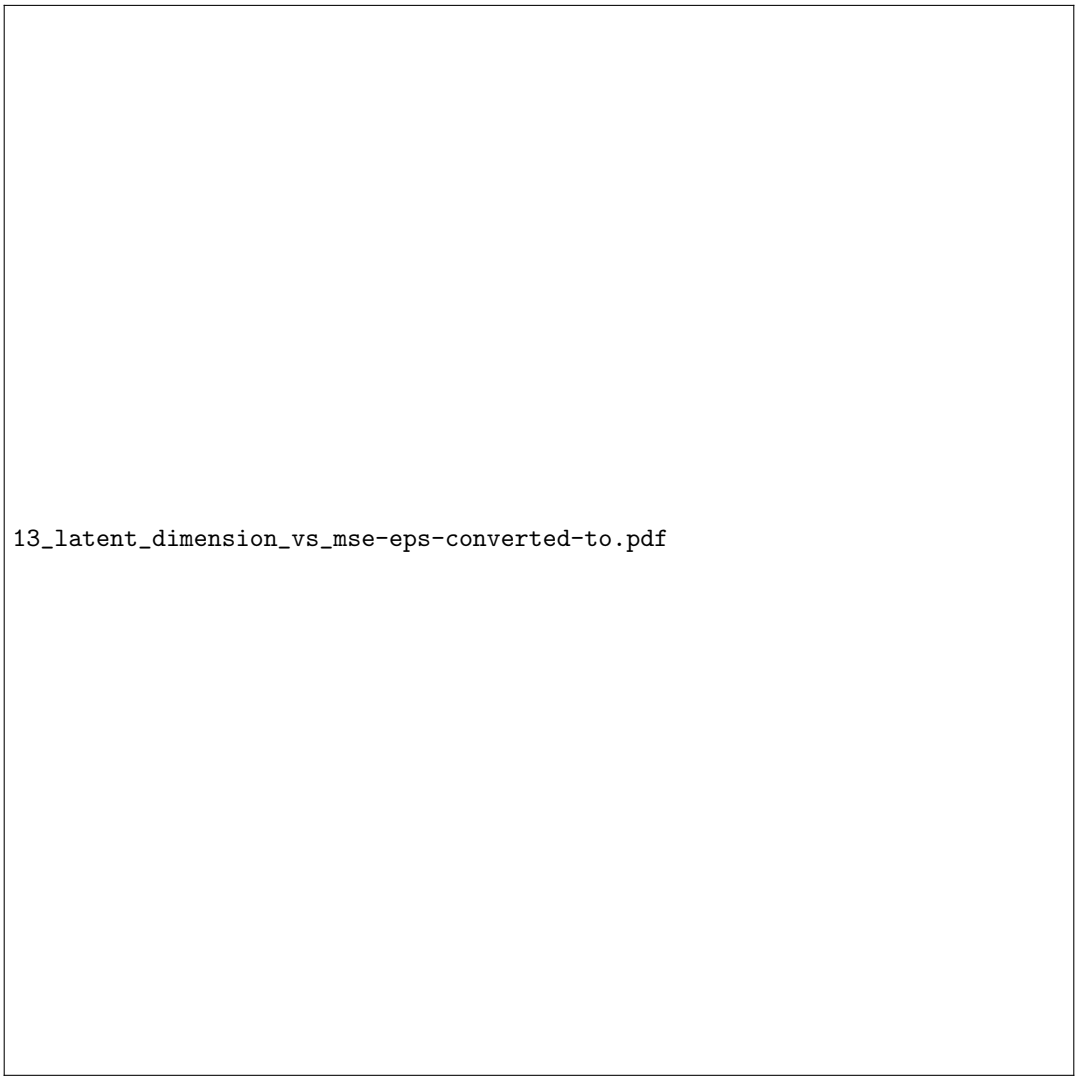


Figure 12: Effect of latent dimension on reconstruction MSE

Inference: The figure shows how reconstruction MSE changes as the latent dimension is varied. A smaller latent dimension provides stronger compression but may lose important image information, whereas increasing the latent dimension provides greater representational capacity and can improve reconstruction quality.

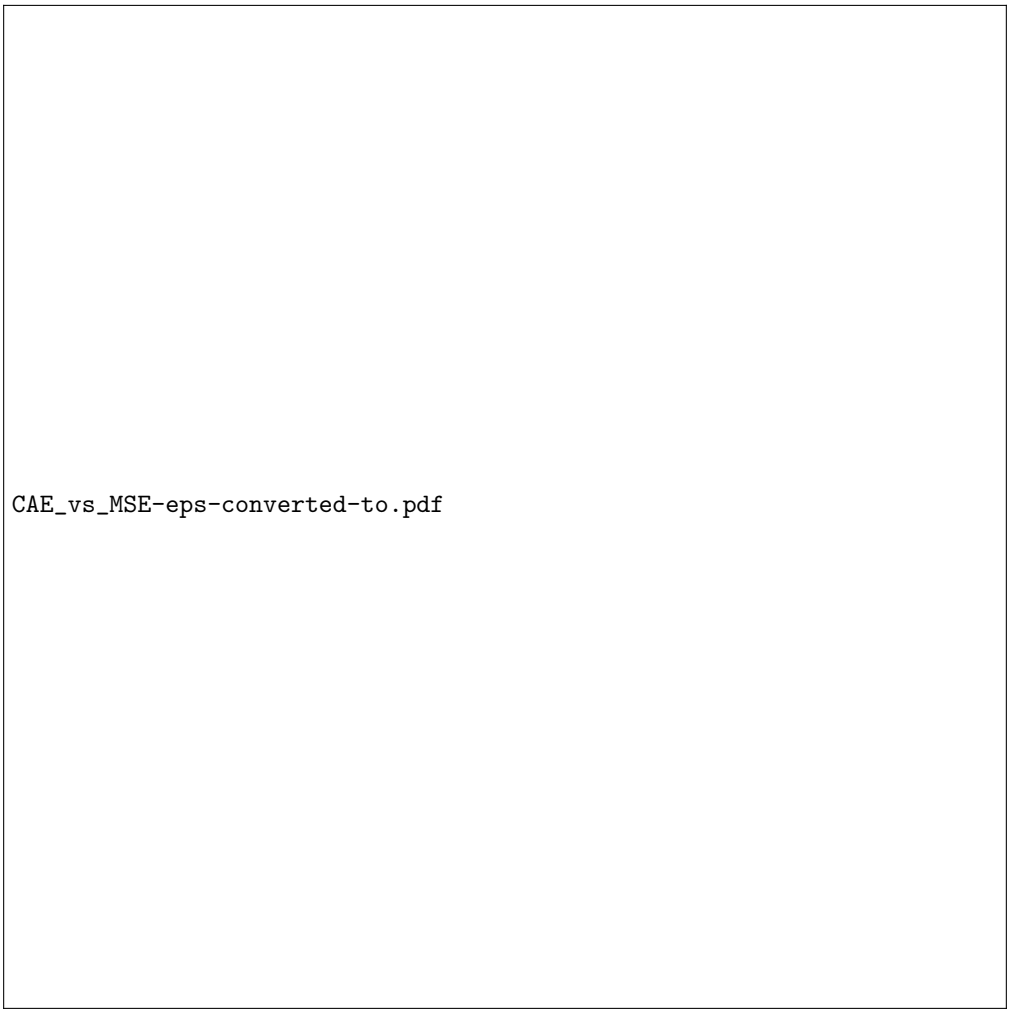


Figure 13: Latent Dimension vs. Reconstruction MSE for CAE

Inference: The plot shows how reconstruction MSE changes with different latent dimensions. It demonstrates the effect of latent-space size on CAE reconstruction quality.



Figure 14: Comparison of Gaussian and Salt-and-Pepper Noise

Inference: The plot compares reconstruction quality for Gaussian and Salt-and-Pepper noise. The results show differences in MSE, MAE, and SSIM between the two noise types.



Figure 15: Effect of Noise Level on Reconstruction Quality

Inference: The plot shows the effect of increasing Gaussian noise level on reconstruction quality. As the noise level changes, the reconstruction metrics indicate the corresponding change in image quality.

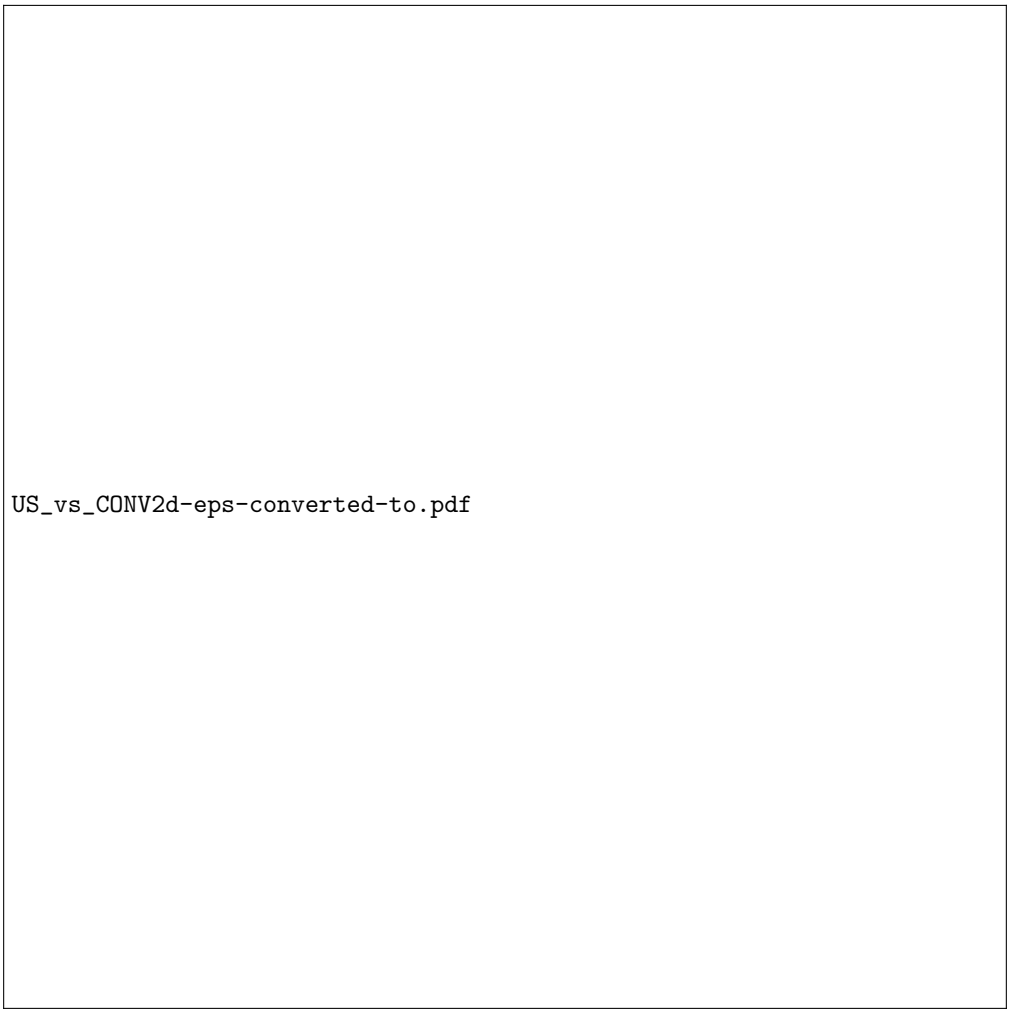


Figure 16: Comparison of UpSampling2D and Conv2DTranspose Decoders

Inference: The plot compares the reconstruction performance of UpSampling2D + Conv2D and Conv2DTranspose decoders using MSE, MAE, SSIM, and training time.

Source Code

GitHub Repository:
`https://github.com/Teena-Pranava/Deep-Learning-Lab`

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

$$\boxed{\text{Image} \rightarrow \text{Encoder} \rightarrow \text{Latent Representation} \rightarrow \text{Decoder} \rightarrow \text{Reconstructed Image}}$$

They should also demonstrate denoising:

$$\boxed{\text{Clean Image} \rightarrow \text{Noise} \rightarrow \text{Denoising Autoencoder} \rightarrow \text{Recovered Image}}$$

and variational generation:

$$\boxed{\text{Image} \rightarrow (\mu, \sigma) \rightarrow z \rightarrow \text{Decoder} \rightarrow \text{Generated/Reconstructed Image}}$$

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`
7. scikit-image Documentation: `https://scikit-image.org`